

TuneMatch Product Requirements Document

August 26, 2026

TuneMatch — Song Similarity Recommender

1. Problem

Spotify's Recommendations and Audio Features APIs are no longer available to new apps. There's no simple free tool where you give one song and get back similar songs you might not already know, with links to listen wherever you want.

2. Goal (v1)

A website where a user types in a song (title + artist), and gets back a list of similar tracks — each with cover art, why it was picked, a 30-second preview, and links to open it on Spotify / Apple Music / YouTube Music.

No login, no account-connecting, no paid APIs.

3. User flow

1. User lands on the site, sees a search box.
2. User types a song + artist (with autocomplete/disambiguation if multiple matches).
3. User selects the correct track from resolved candidates.

4. Backend resolves the seed track, builds a candidate pool, ranks by similarity.
5. Frontend shows a ranked list of results: cover art, title, artist, similarity reason (shared tags/genre/related artist), preview player, platform links.

3a. Design direction

The site should feel dynamic and bold — not a generic/templated look. Think strong visual identity: motion/animation on interactions (search, results appearing), an expressive color palette, and a distinctive layout rather than a plain centered-card SaaS template. This gets addressed in detail when we reach the frontend build steps (step 8+ in the build plan), using proper design-system thinking rather than default component styling.

4. Architecture (v1)

```
[React frontend]
  |
  v
[FastAPI backend]
  |
  +--> Deezer / iTunes Search API  (resolve seed track, previews, cover art)
  +--> Last.fm API                 (tags, similar artists)
  +--> MusicBrainz API             (artist/release relationships)
  +--> Odesli (song.link) API      (cross-platform links)
  |
  v
[Ranking logic: tag overlap + similar-artist graph -> score -> top N]
  |
  v
[Response to frontend]
```

No database in v1 — everything computed per-request. Caching layer is a v2 item.

5. Data sources (all free, no paid tier)

Source	Purpose	Auth
Deezer API	track resolution, cover art, 30s preview	none
iTunes Search API	fallback resolution, previews	none

Source	Purpose	Auth
Last.fm API	tags, similar artists	free key
MusicBrainz API	artist/release relationships	none (rate-limited, need user-agent header)
Odesli (song.link) API	cross-platform links	none

6. Ranking logic (v1, no audio analysis)

Score each candidate by: - Tag overlap with seed track's top Last.fm tags (weighted by tag rank) - Whether the candidate's artist appears in the seed artist's Last.fm "similar artists" list (weighted by similarity score Last.fm provides) - Small penalty for exact same artist as seed (avoid just recommending their discography)

Return top N (default 10), deduplicated by artist+title.

7. Version roadmap

v1 — Core recommender (metadata-only) (*this build*) - Search a song, get ranked recommendations from tags + similar-artist data - Preview playback + cross-platform links - Deployed as a live website (free tier hosting) - No database — stateless, computed per request

v2 — Smarter similarity + caching - Essentia audio feature extraction (tempo, energy, mood) run server-side on preview clips, blended into the ranking score alongside tags - Database/cache layer (e.g. Postgres or a vector DB) so previously-analyzed tracks don't get reprocessed — makes repeat lookups fast and builds up a growing local catalog of analyzed songs over time

v3 — Personalization - Spotify OAuth login (connect your account) - Pull user's saved/top/recently-played tracks to build a taste profile - Bias recommendations toward the user's actual listening history, not just the single seed song

v4 — Agent + memory - Recommendation logic becomes an agent: decides per-request which signal to trust most (tags vs. audio features vs. taste profile) instead of a fixed formula -

Feedback loop: user can like/skip results, stored and retrieved (RAG-style) so future recommendations improve based on accumulated feedback

v5 — Stretch goals - YouTube Music library import (personal-use auth via `ytmusicapi`) - Apple Music support if a developer account is available - Multi-user public mode with per-user accounts/sessions

| 8. Tech stack

v1 - Backend: Python 3, FastAPI, Uvicorn (ASGI server) - HTTP client: `httpx` (async calls to Deezer/iTunes/Last.fm/MusicBrainz/Odesli) - Frontend: React (Vite build), plain CSS (no paid UI kits) - Data layer: none — stateless, computed per request - Env/config: `.env` file for the Last.fm API key (never committed — see `.gitignore`) - Version control: Git, pushed step by step per the build plan below - Hosting (free tier): backend on Render or Railway free tier; frontend on Vercel or Netlify free tier — exact choice decided at deployment step - Testing: `pytest` for backend unit tests on ranking/scoring logic

v2 additions - Essentia (audio analysis library, Python bindings) - Database: Postgres (e.g. free tier on Supabase/Railway/Neon) for caching analyzed tracks — or SQLite locally before choosing a hosted option - Background job handling for audio processing (avoid blocking requests) — candidate: FastAPI `BackgroundTasks` first, Celery/Redis if load needs it

v3 additions - Spotify OAuth (`spotipy` or direct OAuth calls) for account connection - Session/token storage (extends the Postgres DB from v2)

v4 additions - Vector store for feedback/RAG memory (e.g. Chroma — free, self-hosted) or pgvector extension on the existing Postgres DB - Agent orchestration: lightweight, likely hand-rolled tool-selection logic rather than a heavy framework, to keep hosting free-tier friendly

v5 additions - `ytmusicapi` (unofficial, personal-use only) for YouTube Music - Apple MusicKit JS, if a paid Apple Developer account is set up (not free — flagged as a cost decision if we get here) - Auth/session system for multi-user mode (e.g. FastAPI + JWT, or Auth0 free tier)

| 9. Build plan (incremental git commits, v1 only)

1. Repo scaffold + PRD (this commit)
2. Backend: FastAPI skeleton + health check endpoint
3. Backend: Deezer/iTunes track resolution endpoint
4. Backend: Last.fm tag + similar-artist integration
5. Backend: MusicBrainz integration
6. Backend: ranking/scoring logic + `/recommendations` endpoint
7. Backend: Odesli cross-platform link enrichment
8. Frontend: scaffold + search UI
9. Frontend: results list + preview player + platform links
10. Wire frontend to backend, end-to-end test
11. README + deployment docs
12. Deploy v1 (backend + frontend on free hosting tiers)

Later versions (v2+) get their own build plans once v1 is live.

| 10. Open questions

- Last.fm API key: user needs to register one (free, instant) — will provide step-by-step when we hit that integration.
- Rate limits: MusicBrainz requires a descriptive User-Agent header and asks for ~1 request/sec — need to respect this in candidate-pool building.